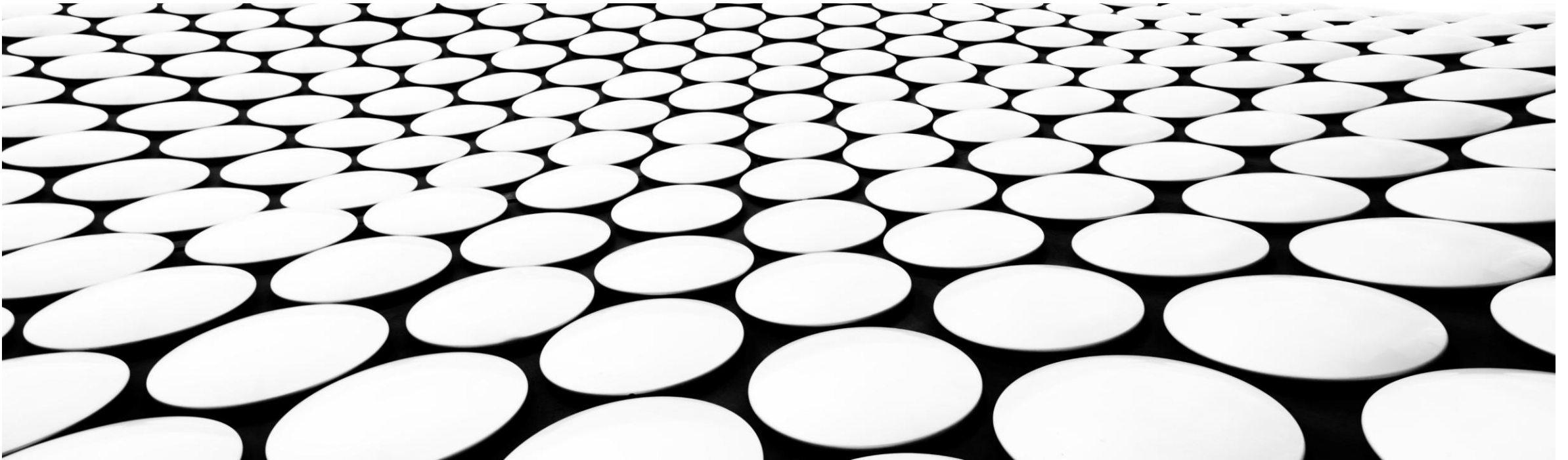

VERTIEFUNG SOFTWARETECHNIK FH-SWF'26

Exemplarischer Softwareentwurf eines Onlinestores

STATE-OF-THE-ART MICROSERVICE BASED APPROACH



Überblick:

- Anforderungen
- Konzept
- Struktur-Grobentwurf
- Prozessablauf (Happy-Path)
- Projektmanagement
- Komponentendetails

Anforderungen:

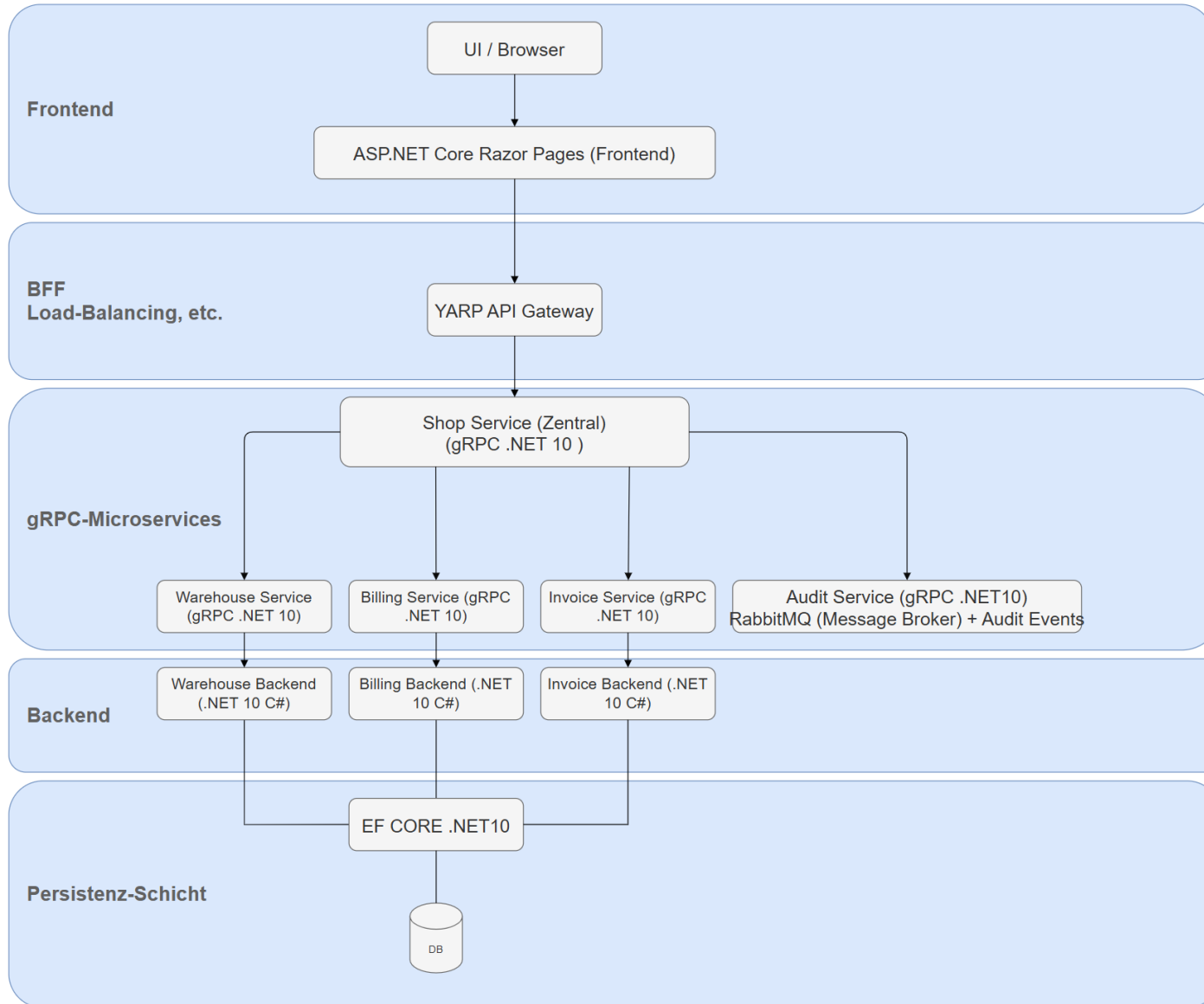
- Es soll ein ‚State-of-the-Art‘ Online-Store mit beliebigem Thema erstellt werden, dabei werden zahlreiche aktuelle Anforderungen an den Entwurf gestellt:
- Nicht-funktionale Anforderungen
 - Wartbarkeit
 - Erweiterbarkeit
 - Nachvollziehbarkeit
 - Resilienz
- Funktionale Anforderungen (Pflicht)
 - Microservice-Architektur
 - Payment-Fassade
 - Fehlerbehandlung (SAGA-Kompensation)
 - Audit- und Snapshot-System
 - Strukturiertes Logging (JSON)
 - Synchrone REST/HTTP Calls von/an die UI (Webpage)
 - Asynchrone Auditing Calls (Services >> AuditService)
 - REST API-Designanforderungen sind zu beachten

Konzept:



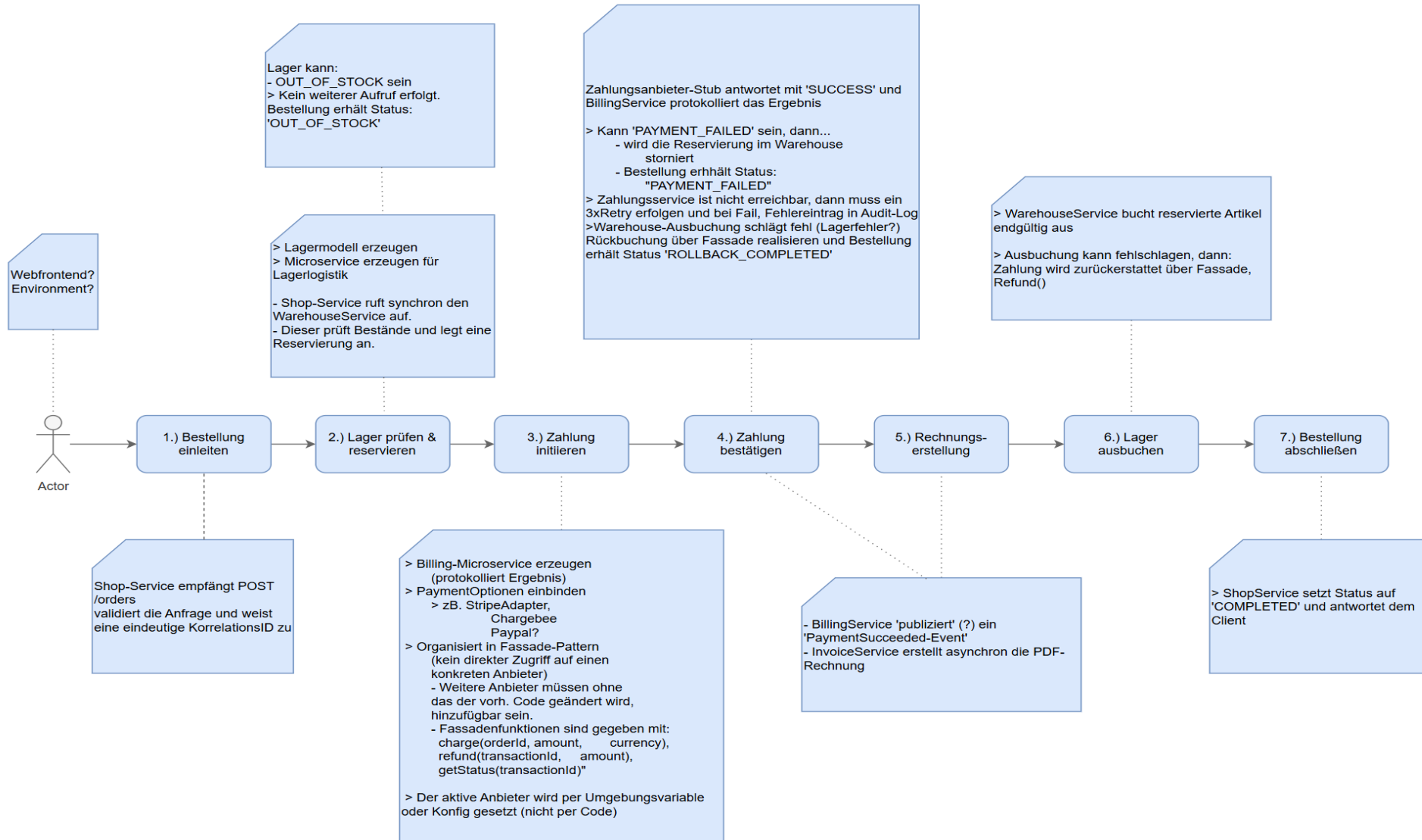
- Das Thema des Online-Stores ist Holz
- Planung für UI:
 - Website und UI Logik – ASP.NET CORE RAZOR-PAGES {.cshtml, .cshtml.cs, .cs}
- API Gateway (BFF) (UI → BFF → gRPC-Main-Service)
 - YARP-Framework – Routing, Load-Balancing { .cs, .json }
- Services
 - Shop-, Billing-, Payment-, Invoice-, Warehouse und Audit-Service per gRPC .NET10
{.proto, .cs}
 - Nur der Shop-Service (Mainservice) soll nach außen exponiert sein
(Mögliches Problem im Zusammenspiel mit YARP ggf. müssen alle Services zumindest für diese Komponente sichtbar sein)
- Message-Broker/Event-Bus:
 - RabbitMQ – async Eventlogging & Auditing {RabbitMQ Server, .cs}
- Service-Backends
 - C# .NET10 Models
 - Ggf. mit EF für DB Anbindung

Struktur-Grobentwurf:



- Schichtenarchitektur (komponentenbasiert, hohe Bindung minimale Kopplung)
- Event-basiertes Auditing (per RabbitMQ Server & Clients)
- Payment-Fassade in Billing-Service (hält mind. 2 Paymentanbieter)
- Shop-Service als DER zentrale Service (managed by YARP-Framework)
- Hohe Response- und Ausführungsgeschwindigkeit
- Fehlerbehandlung nach SAGA-Pattern (Jeder Fehlerfall hat eine eigene Kompensationsoperation)

Prozessablauf (Happy-Path):



Projekt-Management:

- Ansatz: Sowohl Top/Down als auch Bottom/Up, Schnittpunkt zentraler ‚Shop-Service‘
- Umsetzungszeitraum:
 - Juni/Juli: UI / Website / Backend
 - Juli / August: RazorPages / Services
 - August/September YARP BFF / Auditing & Errorhandling / „Hochzeit“ (BFF <-> ShopService <-> Services)



Komponentendetails:

■ gRPC

- **gRPC** ist ein modernes Kommunikationsprotokoll für Anwendungen und Microservices. Es wurde von [Google gRPC Project](https://de.wikipedia.org/wiki/GRPC) entwickelt und wird häufig verwendet, wenn verschiedene Backend-Dienste schnell und typischerweise miteinander kommunizieren sollen. <https://de.wikipedia.org/wiki/GRPC>

■ YARP

- Steht für **Yet Another Reverse Proxy** und ist ein Reverse-Proxy-Framework von [Microsoft YARP Project](https://dotnet.github.io/yarp/) für ASP.NET Core. Mit YARP können Anfragen an verschiedene Backend-Services weitergeleitet werden, ohne selbst viel Routing-Code schreiben zu müssen. <https://dotnet.github.io/yarp/>

■ RabbitMQ

- **RabbitMQ** ist eine weit verbreitete, quelloffene Software für den Nachrichtenaustausch (ein sogenannter *Message Broker*). Die Software fungiert als eine Art intelligentes Postamt für Computersysteme: Sie nimmt Nachrichten (Daten, Aufgaben oder Befehle) von einer Anwendung entgegen, speichert sie sicher und leitet sie an andere Anwendungen weiter. <https://www.rabbitmq.com/>

■ SAGA-Pattern

- **Saga** ([englisch](https://de.wikipedia.org/wiki/Saga_(Entwurfsmuster)) *saga pattern*) ist ein [Entwurfsmuster](https://de.wikipedia.org/wiki/Saga_(Entwurfsmuster)) aus dem Bereich der [Softwareentwicklung](https://de.wikipedia.org/wiki/Saga_(Entwurfsmuster)) zur Verwaltung von Daten in [verteilten Systemen](https://de.wikipedia.org/wiki/Saga_(Entwurfsmuster)), häufig genutzt bei der Implementierung von [Microservices](https://de.wikipedia.org/wiki/Saga_(Entwurfsmuster)). Saga kann in einem verteilten System die [Transaktion](https://de.wikipedia.org/wiki/Saga_(Entwurfsmuster)) ersetzen. [https://de.wikipedia.org/wiki/Saga_\(Entwurfsmuster\)](https://de.wikipedia.org/wiki/Saga_(Entwurfsmuster))

■ Facade-Pattern

- **Fassade** ([englisch](https://de.wikipedia.org/wiki/Fassade_(Entwurfsmuster)) *facade*, auch *façade* geschrieben) ist ein [Entwurfsmuster](https://de.wikipedia.org/wiki/Fassade_(Entwurfsmuster)) aus dem Bereich der [Softwareentwicklung](https://de.wikipedia.org/wiki/Fassade_(Entwurfsmuster)), das zur Kategorie der [Strukturmuster](https://de.wikipedia.org/wiki/Fassade_(Entwurfsmuster)) (engl. *structural design patterns*) gehört. Es bietet eine einheitliche und meist vereinfachte [Schnittstelle](https://de.wikipedia.org/wiki/Fassade_(Entwurfsmuster)) zu einer Menge von Schnittstellen eines [Subsystems](https://de.wikipedia.org/wiki/Fassade_(Entwurfsmuster)). [https://de.wikipedia.org/wiki/Fassade_\(Entwurfsmuster\)](https://de.wikipedia.org/wiki/Fassade_(Entwurfsmuster))

Vielen Dank für Ihre Zeit

KRITIK / FRAGEN / ANREGUNGEN

Philipp Förderer (philipp.foerderer@fh-swf.de)

